

STAT 4710: Homework 5

Put your name here

Due: December 8, 2025 at 11:59am

Contents

Instructions	1
Case Study: Classifying Everyday Objects in CIFAR-10	3
1 Exploratory data analysis (18 points)	3
2 Logistic regression (30 points)	5
3 Multi-layer neural network (26 points)	8
4 Convolutional neural network (26 points)	9

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy,

Please disclose all classmates with whom you collaborated: NA

Please disclose which AI tools you used, and how you used them: ChatGPT for R code debug

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use `homework-5-template.Rmd` as a starting point for your writeup, adding your solutions after `***Solution***`. Add your R code using code chunks and add your text answers using `**bold text**`. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to “print a tibble”, you should print the tibble directly.

Programming

We encourage you to follow the `tidyverse` paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it’s recommended that you:

- Use `pandas` for data manipulation (preferred, since it parallels `dplyr`).
- Use `plotnine` for visualization (preferred, since it parallels `ggplot2`). If using `seaborn`/`matplotlib`, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

Code completion. In some problems, the task is to complete an incomplete code chunk provided or simply to run a given code chunk. In those problems, in the template, there is `eval = FALSE` in the chunk header to prevent evaluation errors. You should remove `eval = FALSE` after you have completed the code to enable evaluation and printing of results.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

Case Study: Classifying Everyday Objects in CIFAR-10

```
tf_function <- function(img) {
  x <- torch_tensor(aperm(img, c(3,1,2)), dtype = torch_float()) / 255
  m <- torch_tensor(cifar10_mean)$view(c(-1,1,1))
  s <- torch_tensor(cifar10_std)$view(c(-1,1,1))
  return((x - m) / s)
}

cifar10_mean <- c(0.4914, 0.4822, 0.4465)
cifar10_std <- c(0.2470, 0.2435, 0.2616)

train_ds_full <- cifar10_dataset("./data", train = TRUE, download = TRUE, transform = tf_function)

## Dataset <cifar10_dataset> (~160 MB) will be downloaded and processed if not
## already available.
## Dataset <cifar10_dataset> loaded with 50000 images across 11 classes.

test_ds <- cifar10_dataset("./data", train = FALSE, download = TRUE, transform = tf_function)

## Dataset <cifar10_dataset> (~160 MB) will be downloaded and processed if not
## already available.
## Dataset <cifar10_dataset> loaded with 10000 images across 11 classes.

set.seed(4710)
n <- length(train_ds_full)
idx <- sample.int(n)
n_train <- floor(0.8 * n)
train_idx <- idx[1:n_train]
valid_idx <- idx[(n_train + 1):n]

batch_size <- 128
train_dl <- dataloader(dataset_subset(train_ds_full, train_idx), batch_size = batch_size)
valid_dl <- dataloader(dataset_subset(train_ds_full, valid_idx), batch_size = batch_size)
test_dl <- dataloader(test_ds, batch_size = batch_size, shuffle = FALSE)

class_names <- c("airplane", "automobile", "bird", "cat", "deer",
                 "dog", "frog", "horse", "ship", "truck")
```

1 Exploratory data analysis (18 points)

1. Solution.

```
# check the proportion of classes
total_sample = length(train_ds_full)
# print proportion of each class
print(table(train_ds_full$y) / total_sample)
```

```
##
## 1 2 3 4 5 6 7 8 9 10
## 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1
```

Answer: We find all classes have the same proportion (10 percent) in the whole dataset.

2. Solution.

```
# check the dimension of images in the training data
sample <- train_ds_full[1]
str(sample)
```

```
## List of 2
## $ x:Float [1:3, 1:32, 1:32]
## $ y: int 7
```

Answer: Each image has size 3x32x32

3. Solution. Answer: From a high level point of view, the `tf_function` normalizes the RGB values of an input image based on mean and standard deviation of cifar10 dataset. In specific, the first line of code reshapes the image to Channel x Height x Width, convert it to a torch tensor, and divide by 255 so each RGB value is between 0 and 1. The second and third line then reshape the mean and standard deviation of cifar10 dataset to the same shape as data tensor and normalize the data by abstracting the mean and dividing the standard deviation

4. Solution.

```
tensor_to_raster <- function(x_chw, # transformed data in CHW format
                             mean = cifar10_mean,
                             std = cifar10_std) {
  m <- torch_tensor(cifar10_mean)$view(c(-1,1,1))
  s <- torch_tensor(cifar10_std)$view(c(-1,1,1))
  x_hwc <- (x_chw * s + m)$permute(c(2,3,1))
  a <- pmin(pmax(as.array(x_hwc), 0), 1)
  cols <- grDevices::rgb(a[,1], a[,2], a[,3]) # convert to colors
  dim(cols) <- dim(a)[1:2]
  return(grDevices::as.raster(cols))
}
```

5. Solution.

```
plot_cifar_grid_ggplot <- function(ds, idxs = 1:6, ncol = 3) {

  plots <- lapply(idxs, function(i) {
    # get the i-th sample from the dataset
    ex <- ds[i]
    img_array <- as.array(ex$x) # get the 3 x 32 x 32 image
    lab <- as.integer(ex$y)
    raster_img <- tensor_to_raster(img_array)
  })
}
```

```

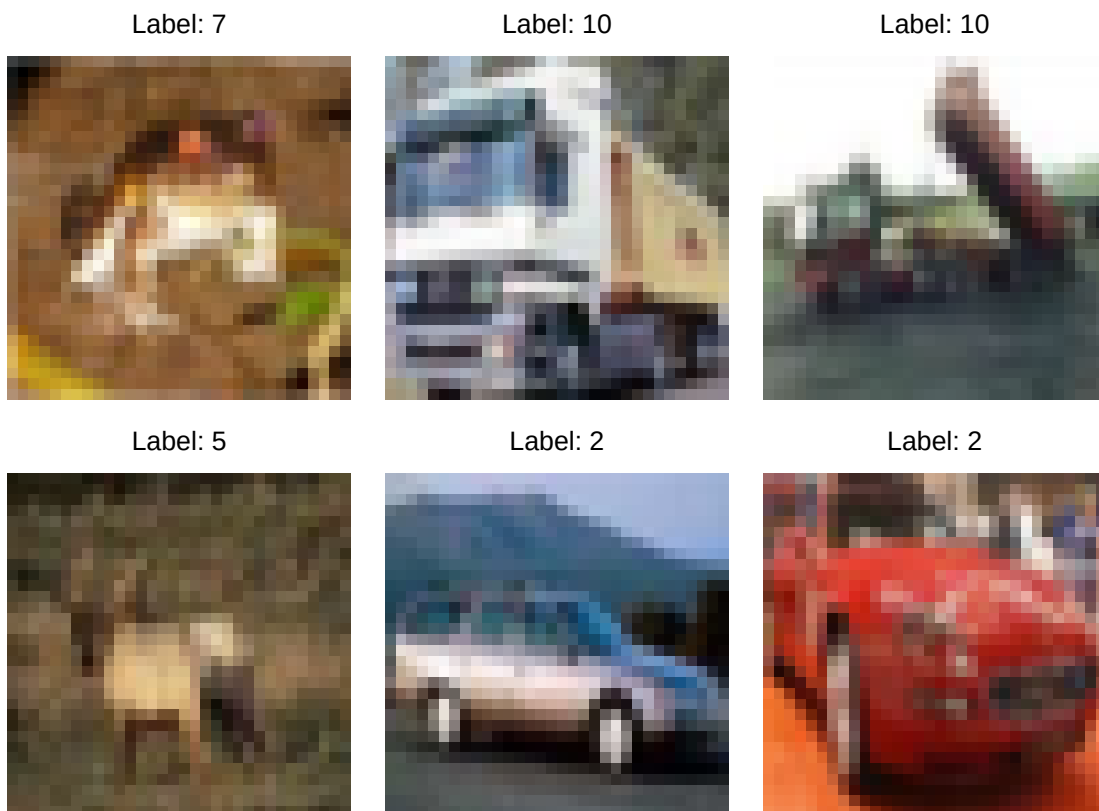
# Create ggplot
ggplot() +
  annotation_raster(raster_img, xmin = 0, xmax = 1, ymin = 0, ymax = 1) +
  labs(title = paste("Label:", lab)) +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5, size = 10)) +
  coord_fixed()
})

# Arrange plots in grid
wrap_plots(plots, ncol = ncol)
}

```

6. Solution.

```
plot_cifar_grid_ggplot(train_ds_full)
```



2 Logistic regression (30 points)

1. Solution. Since each image has size $32 \times 32 \times 3$, when it is flattened, it becomes a single vector with 3072 components in it. So its dimension would be 3072
2. Solution. Since there are 10 classes in the label, the output will be a 10 dimensional vector containing logits for each class.
3. Solution.

```

LogisticRegression <- nn_module(
  initialize = function() {
    # flatten the input vector
    self$flatten <- nn_flatten()
    # define shape for fc layer as 3072 x 10
    self$fc_layer <- nn_linear(3072, 10)
  },
  forward = function(x) {
    # forward pipeline
    x |> self$flatten() |> self$fc_layer()
  }
)

```

4. Solution.

```

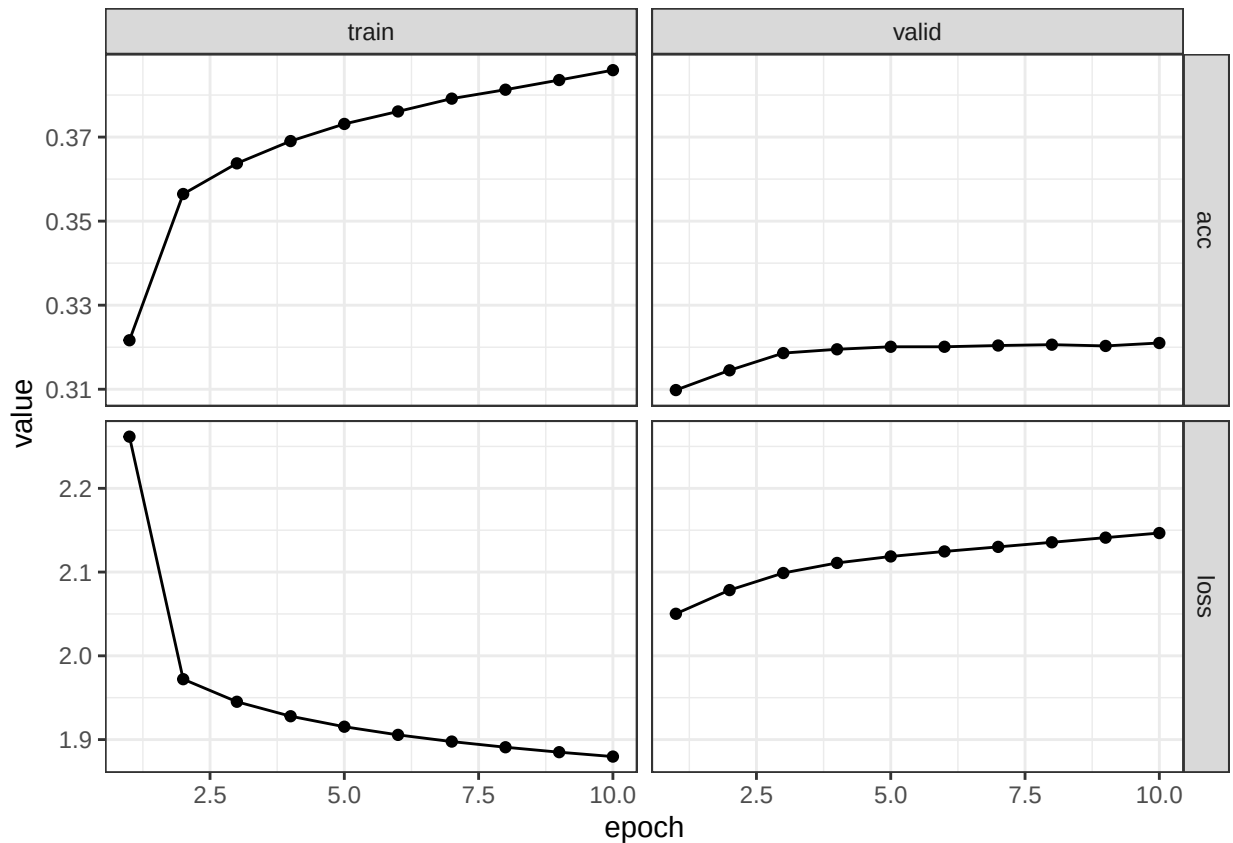
set.seed(4710) # for reproducibility

fit_logistic <- LogisticRegression %>%
  setup(
    loss = nn_cross_entropy_loss(),
    optimizer = optim_rmsprop,
    metrics = list(luz_metric_accuracy())
  ) %>%
  set_opt_hparams(
    lr = 1e-3
  ) %>%
  fit(
    train_dl, epochs = 10, valid_data = valid_dl
  )

```

Plot the fitted logistic model

```
plot(fit_logistic)
```



5. Solution. From the training curve, the training accuracy continues to rise steadily, while the validation accuracy stops improving after around epoch 5. This divergence indicates that the model is overfitting, since it keeps fitting the training data better but fails to improve on unseen data.

6. Solution.

```
# evaluate the model on test accuracy
evaluate(fit_logistic, test_dl)
```

```
## A 'luz_module_evaluation'
## -- Results -----
## loss: 2.1535
## acc: 0.3191
```

Answer: The test accuracy is 0.3191.

7. Solution. Since the input vector has 3072 components, its number of parameters are computed by $(3072+1) \times 10 = 30730$ parameters, where the plus one means the bias term and 10 is the number of classes. This result also agrees with the output below.

```
count_params <- function(module) sum(sapply(module$parameters, \(p) as.numeric(p$numel()))
count_params(fit_logistic$model)
```

```
## [1] 30730
```

3 Multi-layer neural network (26 points)

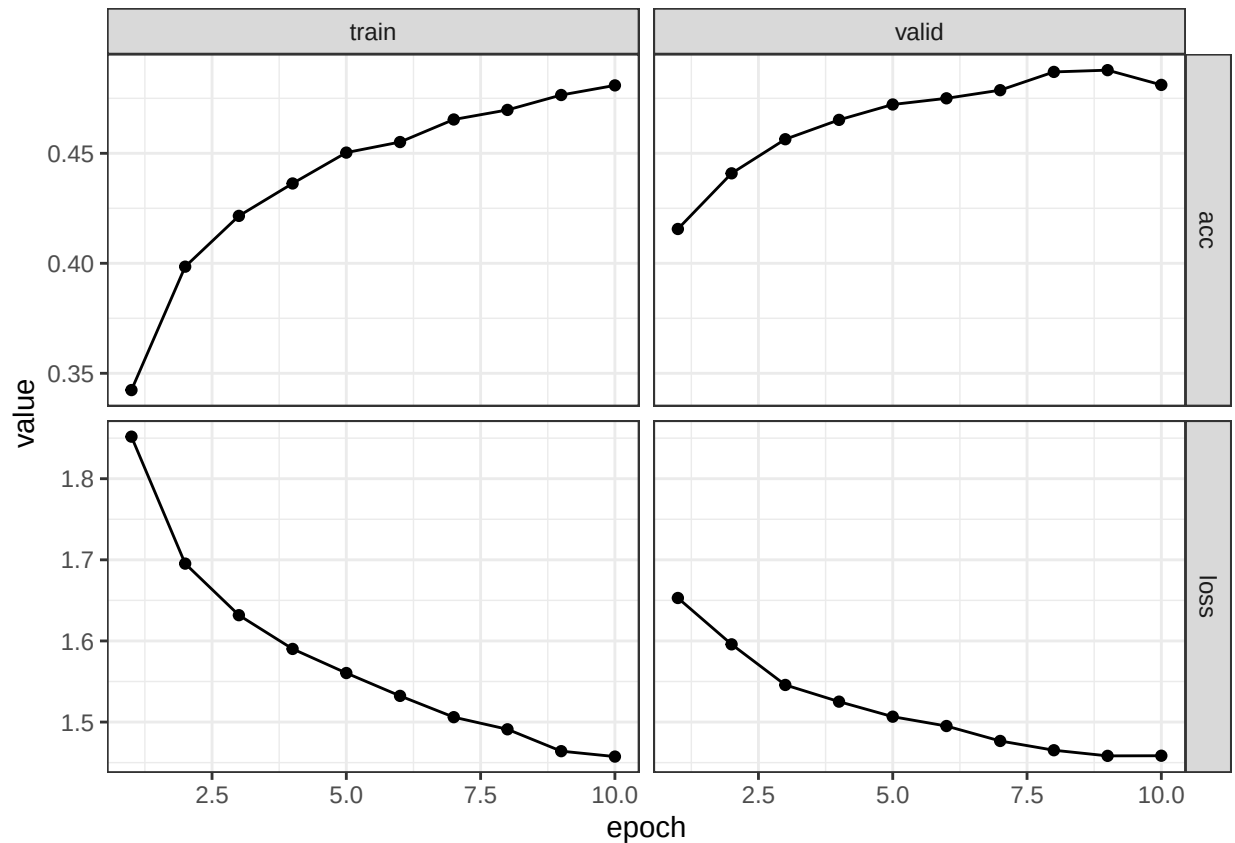
1. Solution. Since the multi-layer structure only affects the intermediate layers, the input and output dimension of NN will remain the same: a 3072 dimensional vector as input and a 10 dimensional vector as output.

2. Solution.

```
MLP <- nn_module(  
  initialize = function() {  
    self$flatten <- nn_flatten()  
    self$fc1 <- nn_linear(3072, 256)  
    self$drop1 <- nn_dropout(0.4)  
    self$fc2 <- nn_linear(256, 128)  
    self$drop2 <- nn_dropout(0.3)  
    self$fc3 <- nn_linear(128, 10)  
  },  
  forward = function(x) {  
    x <- self$flatten(x) # x$view(base::c(x$size(1), P))  
    x <- nnf_relu(self$fc1(x)) # first hidden layer  
    x <- self$drop1(x)  
    x <- nnf_relu(self$fc2(x)) # second hidden layer  
    x <- self$drop2(x)  
    self$fc3(x) # logits  
  }  
)
```

3. Solution.

```
set.seed(4710) # for reproducibility  
fit_mlp <- MLP %>%  
  setup(  
    loss = nn_cross_entropy_loss(),  
    optimizer = optim_adam,  
    metrics = list(luz_metric_accuracy())  
  ) %>%  
  set_opt_hparams(  
    lr = 1e-3  
  ) %>%  
  fit(  
    train_dl, epochs = 10, valid_data = valid_dl,  
    callbacks = list(  
      luz_callback_early_stopping(patience = 3, monitor = "valid_loss"),  
      luz_callback_model_checkpoint(  
        path = "checkpoints/best.pt",  
        monitor = "valid_loss", save_best_only = TRUE  
      )  
    )  
  )  
plot(fit_mlp)
```

4. **Solution.** Based on the training curve, we observe both training accuracy and validation accuracy increase, without reaching a plateau. This indicates the model is still in the learning process and therefore is underfitting and need more epochs during the training stage.

5. **Solution.**

```
# evaluate the model on test accuracy
evaluate(fit_mlp, test_dl)
```

```
## A 'luz_module_evaluation'
## -- Results -----
## loss: 1.4404
## acc: 0.4923
```

Answer: The test accuracy is 0.4915.

4 Convolutional neural network (26 points)

1. **Solution.** For this NN, the dimension of the input will be a 3x32x32 tensor and the output will be 10 dimensional vector.

2. **Solution.**

```

SmallCNN <- nn_module(
  initialize = function() {
    self$conv1 <- nn_conv2d(in_channels=3, out_channels=32, kernel_size=3)
    self$conv2 <- nn_conv2d(32, 64, kernel_size=3)
    self$pool <- nn_max_pool2d(kernel_size=2)
    self$drop1 <- nn_dropout(0.25)
    self$flatten <- nn_flatten()
    self$fc1 <- nn_linear(64*14*14, 256)
    self$drop2 <- nn_dropout(0.5)
    self$fc2 <- nn_linear(256, 10)
  },
  forward = function(x) {
    x <- nnf_relu(self$conv1(x))
    x <- nnf_relu(self$conv2(x))
    x <- self$pool(x)
    x <- self$drop1(x)
    x <- self$flatten(x) # flatten [B,64,14,14] -> [B,12544]
    x <- nnf_relu(self$fc1(x))
    x <- self$drop2(x)
    self$fc2(x)
  }
)

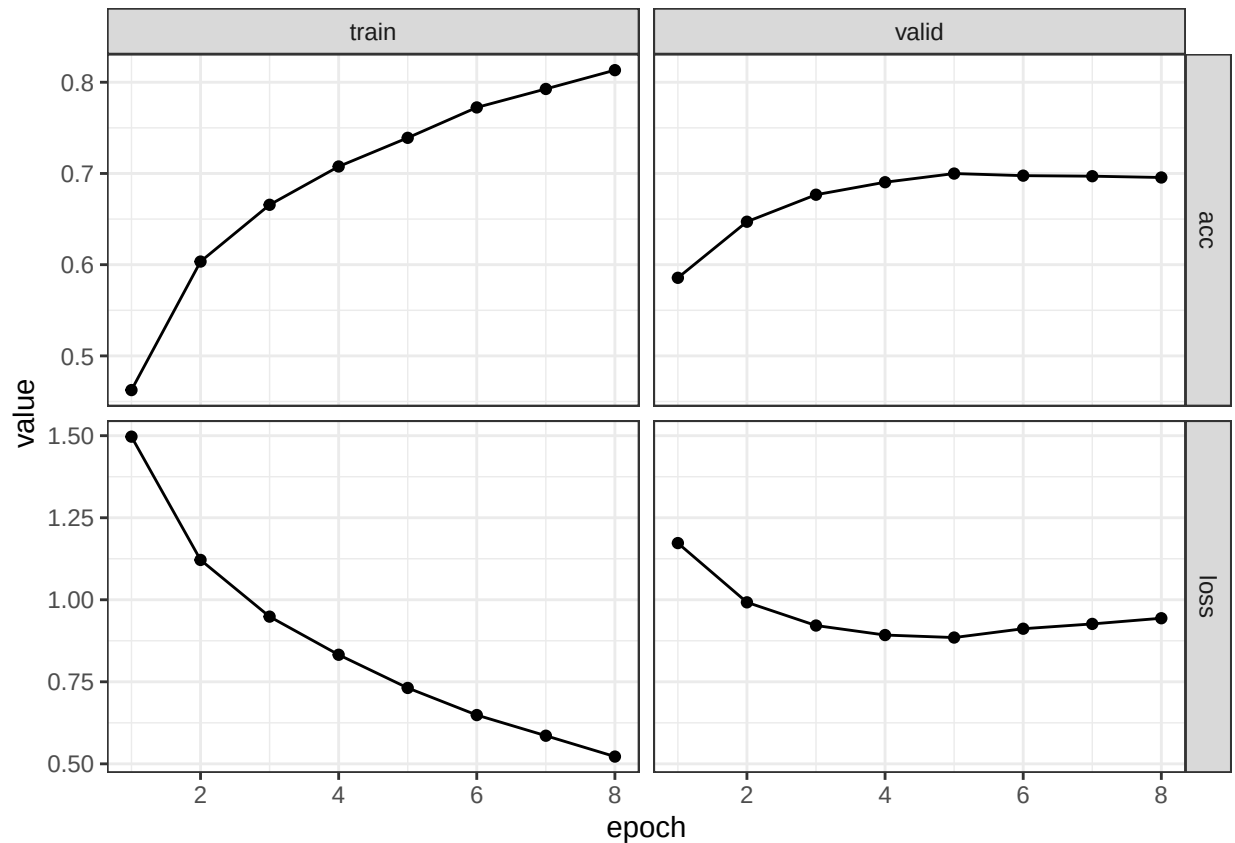
```

3. Solution.

```

set.seed(4710) # for reproducibility
fit_cnn <- SmallCNN %>%
  setup(
    loss = nn_cross_entropy_loss(),
    optimizer = optim_adam,
    metrics = list(luz_metric_accuracy())
  ) %>%
  set_opt_hparams(
    lr = 1e-3
  ) %>%
  fit(
    train_dl, epochs = 10, valid_data = valid_dl,
    callbacks = list(
      luz_callback_early_stopping(patience = 3, monitor = "valid_loss"),
      luz_callback_model_checkpoint(
        path = "checkpoints/best_cnn.pt",
        monitor = "valid_loss", save_best_only = TRUE
      )
    )
  )
plot(fit_cnn)

```



4. **Solution.** From the training curve, the training accuracy continues to rise steadily, while the validation accuracy stops improving after around epoch 4. This divergence indicates that the model is overfitting, since it keeps fitting the training data better but fails to improve on unseen data.

5. **Solution.**

```
# evaluate the model on test accuracy
evaluate(fit_cnn, test_dl)
```

```
## A 'luz_module_evaluation'
## -- Results -----
## loss: 0.9466
## acc: 0.7001
```

Answer: the test accuracy for CNN is 0.7069.